An-Najah Nation University

Faculty of Engineering

جامعة النجاح الوطنية كلية الهندسة

Computer Engineering Department

Networks1 (10636454)

Programming HW project

ChatLite — Minimal Real-Time Chat System using Java Sockets

Instructor name:
Dr. Eng. Saed TARAPIAH

Student#1 Name: mohammed fares

Student#2 Name : Tahseen qut

Academic Year: **2025-2026**

Semester: **Spring**

Credit Hours: **3**

Date: **01/April/2026**

Registration Number#1: 12323073.....

Registration Number#2: 12240962.....

Total Exam Mark: **15**

Exam Weight: **15%**

Deadline: 31/March/2026 midnight

Question	Points	ILO's	ILO's%	Question Grade	Required Time
Q1	15				2 Weeks
Student Grade					

Introduction/ Abstract

ChatLite is a real-time chat application built on TCP Persistent connections. We chose TCP over UDP because TCP guarantees ordered, reliable delivery with error checking, while UDP is faster but drops packets without retransmission — not acceptable for a chat system where every message must arrive. Within TCP, we used persistent connections, meaning one connection stays open for the entire session rather than opening and closing a new socket per message (non-persistent). This reduces overhead and latency significantly in a chat scenario. The system supports multi-client communication through chat rooms, private messaging, and user status updates.

Procedure

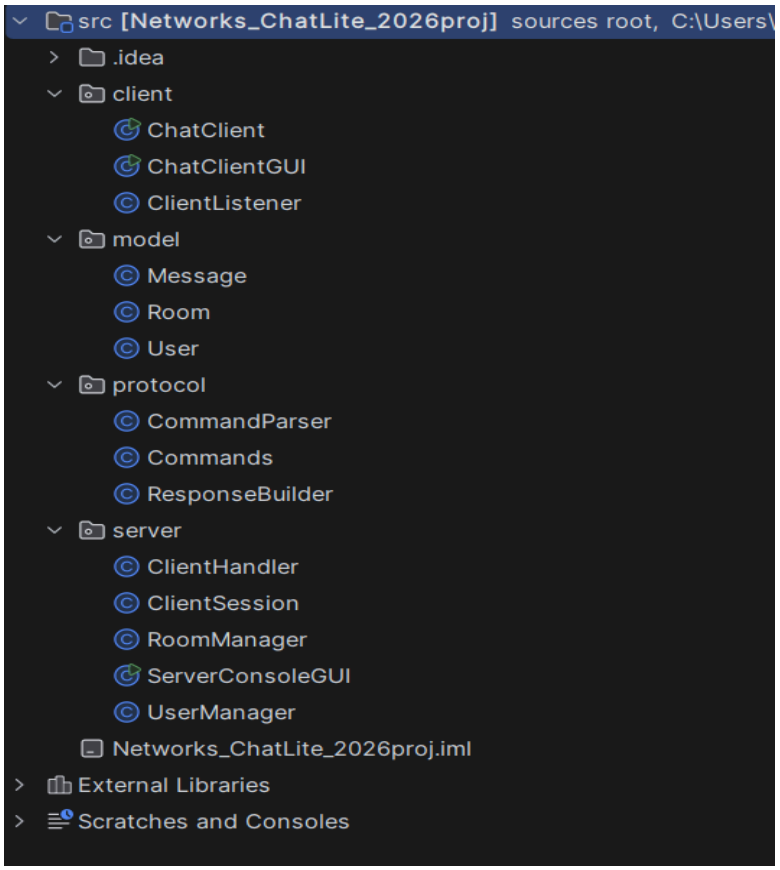
Tools and environment used:

- Java JDK-25
- IntelliJ IDEA as the development environment
- Java Swing for the graphical user interfaces
- Java ServerSocket and Socket classes for TCP communication
- One machine running both server and clients (localhost:5000)

The system was built in the following steps:

- Designed the protocol commands and response codes on paper first
- Implemented the server core: ServerSocket, ClientHandler threads, UserManager, RoomManager
- Built the ServerConsoleGUI to manage users and monitor sessions visually
- Implemented the client: ChatClientGUI with login dialog, chat area, private messaging panel
- Added authentication: admin registers users with passwords before they can connect
- Tested with multiple client instances running simultaneously

System Architecture Diagram:



Results and Calculations

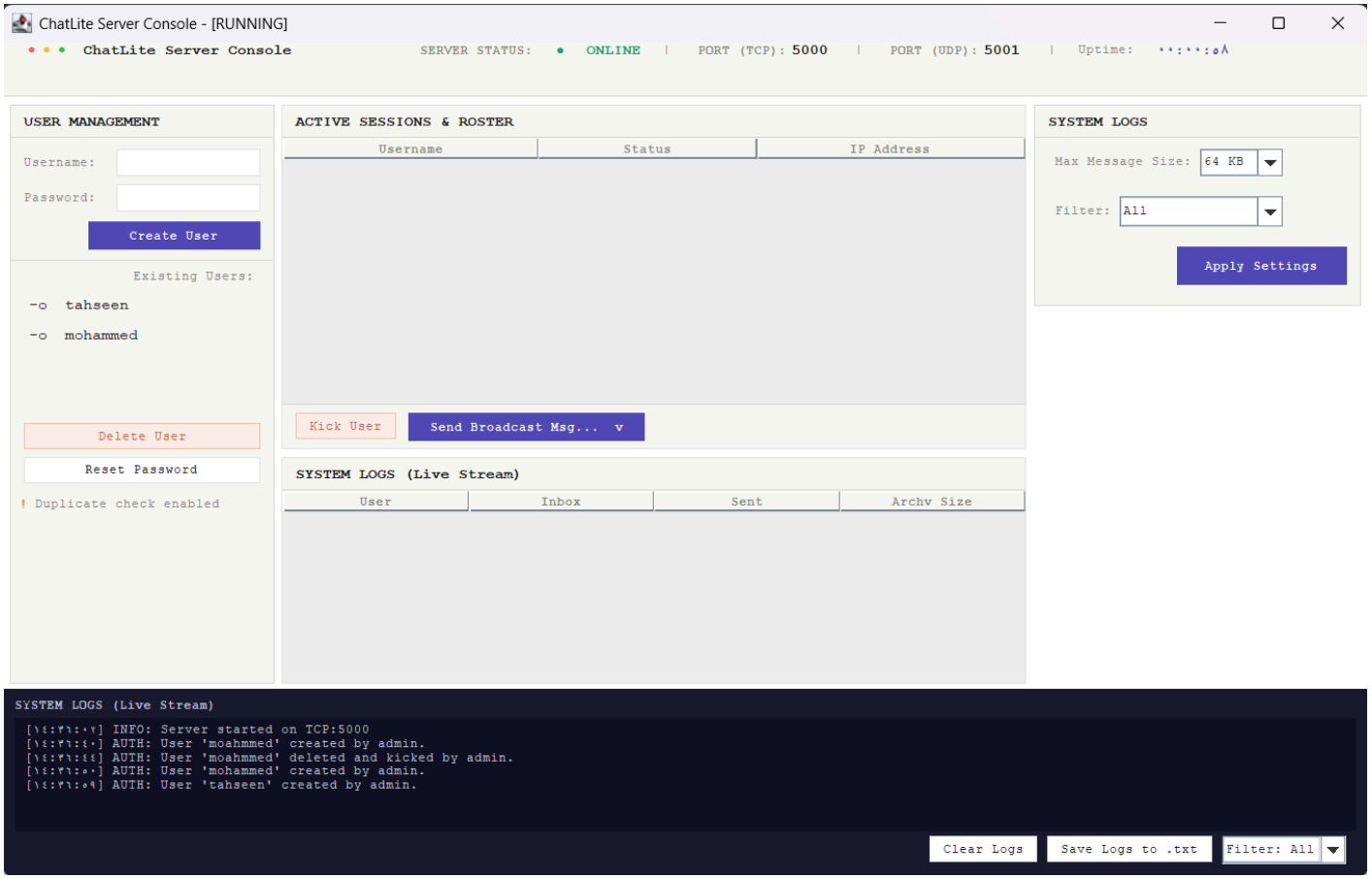
We tested the system and the following behaviors were verified :

- Users could only connect if pre-registered by the admin with a minimum 8-character password
- Messages sent in a room were received by all users in that room in real time
- Private messages were delivered only to the intended recipient
- Changing status (Active/Busy/Away) was reflected in all other clients' user lists
- Kicking a user disconnected them and blocked reconnection until server restart
- Typing QUIT removed the user from the active sessions list on the server

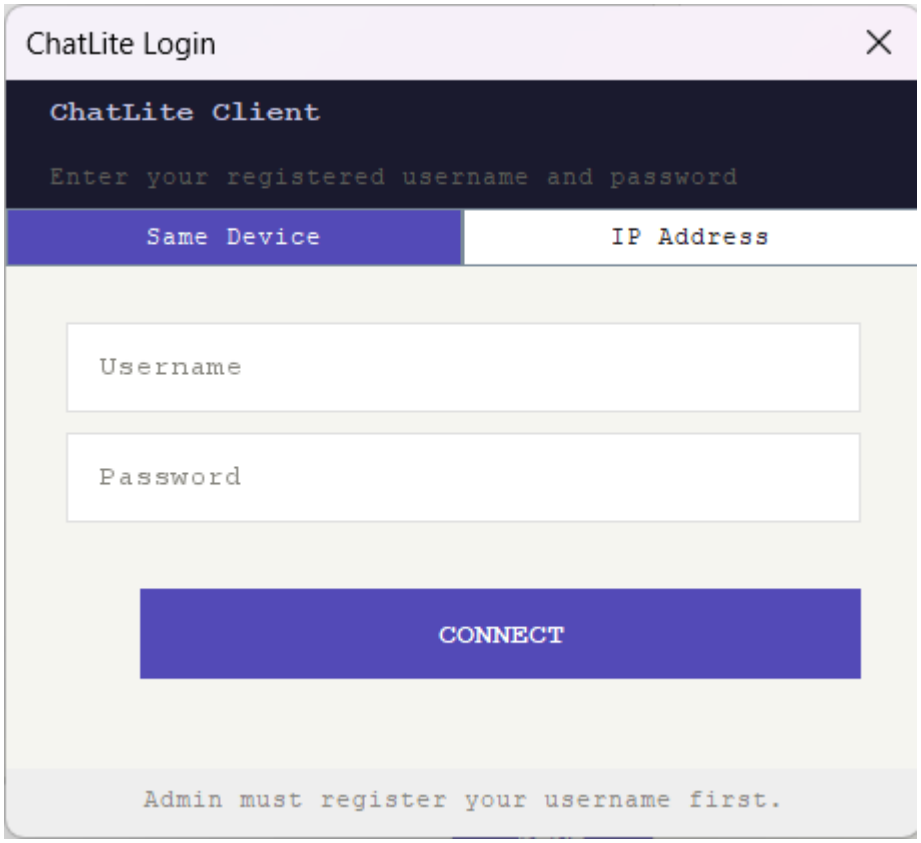
Client sends	Meaning
`JOIN room`	Join a chat room
`MSG room text`	Send message to room
`PM user text`	Send private message
`USERS`	Get user list with status
`ROOMS`	Get room list
`LEAVE room`	Leave a room
`QUIT`	Disconnect and unregister

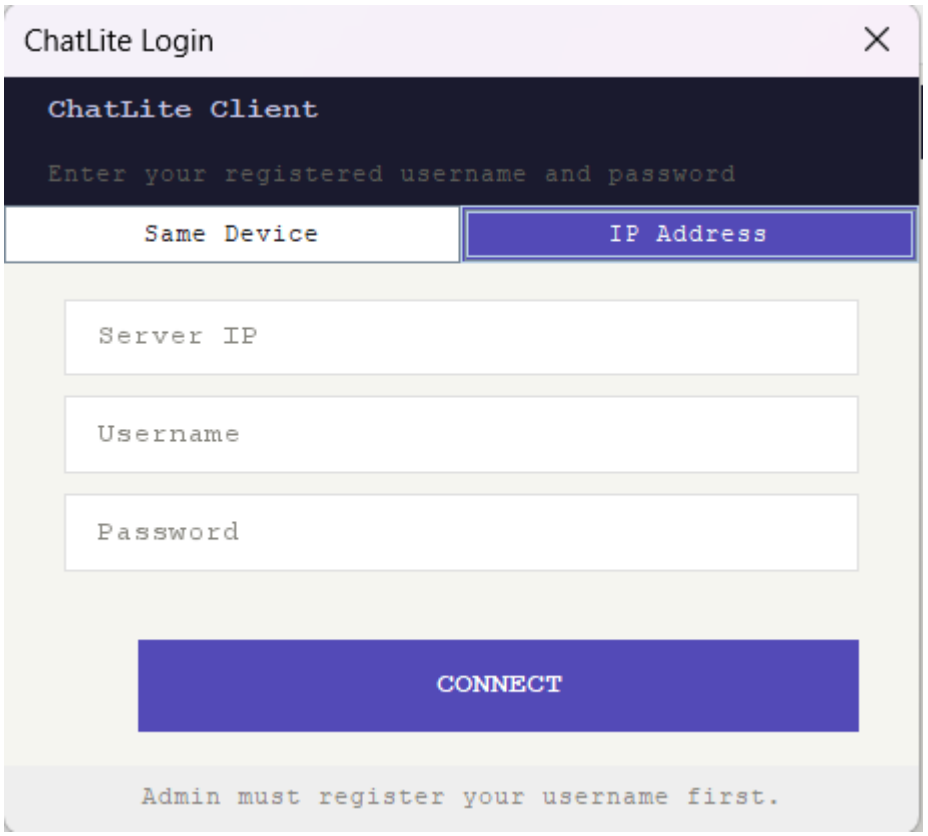
Note : we build the code to be case insenstive so typing join or JOIN giving same results

1. Server Console Screenshot:

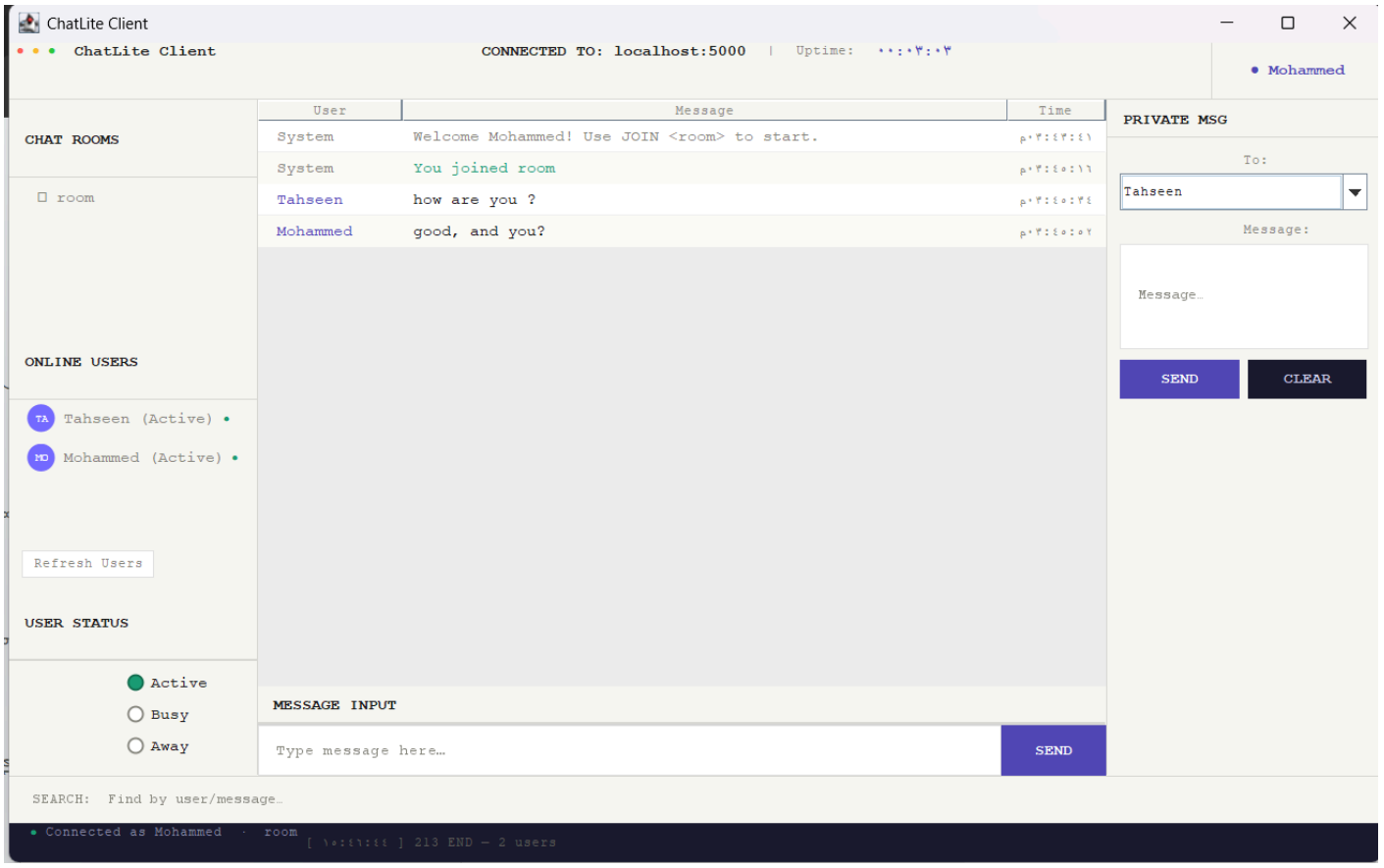


2. Client Interface Screenshot:




ChatLite Login
ChatLite Client
Enter your registered username and password
Same Device
IP Address
Server IP
Username
Password
CONNECT
Admin must register your username first.

Then


ChatLite Client
ChatLite Client
CONNECTED TO: localhost:5000
Mohammed
CHAT ROOMS
room
ONLINE USERS
Tahseen (Active)
Mohammed (Active)
Refresh Users
USER STATUS
Active
Busy
Away
User
Message
Time
System
Welcome Mohammed! Use JOIN <room> to start.
System
You joined room
Tahseen
how are you ?
Mohammed
good, and you?
MESSAGE INPUT
Type message here...
SEND
PRIVATE MSG
To:
Tahseen
Message:
Message...
SEND
CLEAR
SEARCH: Find by user/message...
Connected as Mohammed
room

Discussion

The system performed as expected for the main use cases. Multi-client communication worked reliably, and the protocol was simple enough to implement without issues. A few observations worth mentioning:

The authentication system worked well. Users who were not registered by the admin received a clear error message and could not connect. The password check on both the client and server sides made it hard to bypass, and it also makes sure that the password is at least 8 characters in length.

One limitation we noticed is that passwords and user registrations are stored in memory only, so everything resets when the server restarts. For a production system, this would need to be stored in a database or file.

The simulated IP addresses (192.168.1.X) in the server console are assigned sequentially per session since all connections come from localhost in our test setup. In a real network deployment, these would be the actual client IPs.

The protocol has no encryption, meaning messages travel as plain text over the network. Adding TLS would be a natural next step for a more complete implementation.

Conclusions

We built a working multi-client chat system that shows the main networking ideas in action. The TCP connection was stable, and the program handled multiple users at the same time